ASSIGNMENT- 08

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Basic SQL Queries and Aggregate Functions

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

SAMPLE DATA INSERTION FOR AN ONLINE FOOD DELIVERY SYSTEM

1. Introduction

Sample data insertion is an important step in database implementation. It involves adding realistic records to database tables to test their structure, relationships, and constraints.

In this assignment, sample data is inserted into the ten tables of the Online Food Delivery System developed in the previous assignments. The system manages customers, delivery addresses, restaurants, menu items, orders, order items, delivery agents, deliveries, payments, and reviews.

The SQL `INSERT INTO` command is used to insert sample records into each table. The records are designed to represent a realistic food delivery application while maintaining referential integrity and satisfying the defined constraints.

2. Objectives

1. To insert realistic sample data into all ten database tables.
2. To demonstrate the use of the SQL `INSERT INTO` command.
3. To maintain primary key and foreign key relationships.
4. To ensure that inserted data satisfies UNIQUE and CHECK constraints.
5. To simulate customer registrations, restaurant menus, orders, payments, and deliveries.
6. To verify that the inserted records are stored correctly.

3. Database Used

Database Name: FoodDeliveryDB

The database contains the following tables:

Sl. No.	Table Name	Sample Data
1	Customers	Customer details
2	Addresses	Customer delivery addresses

3	Restaurants	Restaurant information
4	MenuItems	Food items and prices
5	Orders	Customer orders
6	OrderItems	Items included in each order
7	DeliveryAgents	Delivery personnel details
8	Deliveries	Delivery assignments and status
9	Payments	Payment information
10	Reviews	Customer ratings and feedback

4. SQL Implementation

The following SQL statements use MySQL syntax and assume that the ten tables have already been created using the definitions from Assignment 5.

Important: Run these statements in the order shown. The sample records use explicit IDs to make the relationships easy to understand. Execute them in a clean database where these IDs do not already exist.

4.1 Selecting the Database

```
USE FoodDeliveryDB;
```

This command selects the database in which the sample records will be inserted.

4.2 Inserting Data into the Customers Table

The Customers table stores customer names, email addresses, phone numbers, password hashes, and registration dates.

```
INSERT INTO Customers

(customer_id, full_name, email, phone_number,

 password_hash, registration_date)

VALUES

(1, 'Mohammed Sinan', 'sinan@example.com',

 '9876543210', 'hash_sinan_001', '2026-01-10'),



(2, 'Rahul Menon', 'rahul@example.com',

 '9876543211', 'hash_rahul_002', '2026-02-15'),



(3, 'Ananya Nair', 'ananya@example.com',

 '9876543212', 'hash_ananya_003', '2026-03-05'),



(4, 'Arjun Kumar', 'arjun@example.com',

 '9876543213', 'hash_arjun_004', '2026-04-12'),



(5, 'Diya Thomas', 'diya@example.com',

 '9876543214', 'hash_diya_005', '2026-05-20');
```

Explanation: Five customer records are inserted with unique customer IDs, email addresses, and phone numbers.

The password hash values are placeholders for this academic example, not actual secure password hashes.

4.3 Inserting Data into the Addresses Table

The Addresses table stores delivery addresses belonging to registered customers.

```
INSERT INTO Addresses
(address_id, customer_id, address_line, city,
 state, pincode, address_type)
VALUES
(1, 1, 'Main Road, Anekal',
 'Bengaluru', 'Karnataka', '562106', 'Home'),

(2, 1, 'Electronic City Phase 1',
 'Bengaluru', 'Karnataka', '560100', 'Work'),

(3, 2, 'MG Road',
 'Bengaluru', 'Karnataka', '560001', 'Home'),

(4, 3, 'HSR Layout Sector 2',
 'Bengaluru', 'Karnataka', '560102', 'Home'),

(5, 4, 'Whitefield Main Road',
 'Bengaluru', 'Karnataka', '560066', 'Home'),

(6, 5, 'BTM Layout',
 'Bengaluru', 'Karnataka', '560076', 'Home');
```

Explanation: Six addresses are inserted for five customers. Customer 1 has two addresses, demonstrating the one-to-many relationship between Customers and Addresses.

4.4 Inserting Data into the Restaurants Table

The Restaurants table stores restaurant names, cuisine types, addresses, contact numbers, ratings, and active status.

```
INSERT INTO Restaurants
(restaurant_id, restaurant_name, cuisine_type,
 address, phone_number, rating, is_active)
VALUES
(1, 'Food Palace', 'Indian',
 'Anekal, Bengaluru', '9000000001', 4.5, TRUE),

(2, 'Spice Garden', 'South Indian',
 'Electronic City, Bengaluru', '9000000002', 4.2, TRUE),

(3, 'Pizza Corner', 'Italian',
 'HSR Layout, Bengaluru', '9000000003', 4.0, TRUE),

(4, 'Burger Hub', 'Fast Food',
 'Whitefield, Bengaluru', '9000000004', 4.3, TRUE),

(5, 'Sweet Treats', 'Desserts',
 'BTM Layout, Bengaluru', '9000000005', 4.6, TRUE);
```

Explanation: Five restaurants are inserted with ratings between 0 and 5. All restaurants are marked as active.

4.5 Inserting Data into the MenuItems Table

The MenuItems table stores food item names, descriptions, prices, categories, and availability.

```
INSERT INTO MenuItems
(item_id, restaurant_id, item_name, description,
 price, category, is_available)
VALUES
```

(1, 1, 'Veg Burger', 'Fresh vegetable burger',
 120.00, 'Main Course', TRUE),

(2, 1, 'French Fries', 'Crispy potato fries',
 80.00, 'Starter', TRUE),

(3, 2, 'Masala Dosa', 'Dosa with potato masala',
 90.00, 'Main Course', TRUE),

(4, 2, 'Idli', 'Steamed rice cakes',
 50.00, 'Starter', TRUE),

(5, 3, 'Margherita Pizza', 'Classic cheese pizza',
 250.00, 'Main Course', TRUE),

(6, 3, 'Garlic Bread', 'Toasted garlic bread',
 120.00, 'Starter', TRUE),

(7, 4, 'Chicken Burger', 'Grilled chicken burger',
 180.00, 'Main Course', TRUE),

(8, 4, 'Veg Burger', 'Vegetable patty burger',
 110.00, 'Main Course', TRUE),

(9, 5, 'Chocolate Cake', 'Chocolate layer cake',
 150.00, 'Dessert', TRUE),

(10, 5, 'Ice Cream', 'Vanilla ice cream',

70.00, 'Dessert', TRUE);

Explanation: Ten food items are inserted across five restaurants. Each item references an existing restaurant through `restaurant_id`.

All prices are non-negative, satisfying the CHECK constraint.

4.6 Inserting Data into the Orders Table

The Orders table stores customer orders, the restaurant, delivery address, order date, status, and total amount.

INSERT INTO Orders

(order_id, customer_id, restaurant_id, address_id,

order_date, order_status, total_amount)

VALUES

(1, 1, 1, 1, '2026-10-20 12:30:00',

'Delivered', 320.00),

(2, 2, 2, 3, '2026-10-20 13:15:00',

'Delivered', 180.00),

(3, 3, 3, 4, '2026-10-21 19:00:00',

'Delivered', 370.00),

(4, 4, 4, 5, '2026-10-21 20:15:00',

'Out for Delivery', 360.00),

(5, 5, 5, 6, '2026-10-22 10:30:00',

'Preparing', 220.00);

Explanation: Five orders are inserted with different customers, restaurants, addresses, dates, statuses, and totals.

Every customer ID, restaurant ID, and address ID references an existing record.

4.7 Inserting Data into the OrderItems Table

The OrderItems table records the food items and quantities included in each order.

```
INSERT INTO OrderItems
(order_item_id, order_id, item_id,
 quantity, price_at_order)
VALUES
(1, 1, 1, 2, 120.00),
(2, 1, 2, 1, 80.00),

(3, 2, 3, 2, 90.00),

(4, 3, 5, 1, 250.00),
(5, 3, 6, 1, 120.00),

(6, 4, 7, 2, 180.00),

(7, 5, 9, 1, 150.00),
(8, 5, 10, 1, 70.00);
```

Explanation: Eight order-item records are inserted.

For example, Order 1 contains two Veg Burgers and one portion of French Fries.

The subtotal is:

(2 × ₹120) + (1 × ₹80) = ₹320

This matches the total amount recorded for Order 1.

The quantity and price constraints are also satisfied.

4.8 Inserting Data into the DeliveryAgents Table

The DeliveryAgents table stores delivery agent names, phone numbers, vehicle numbers, and availability.

```
INSERT INTO DeliveryAgents

(agent_id, agent_name, phone_number,

 vehicle_number, is_available)

VALUES

(1, 'Arun Kumar', '9100000001',

 'KA01AB1234', TRUE),


(2, 'Vishal Raj', '9100000002',

 'KA03CD5678', TRUE),


(3, 'Kiran Das', '9100000003',

 'KA05EF9012', FALSE),


(4, 'Rohit Sharma', '9100000004',

 'KA51GH3456', TRUE);
```

Explanation: Four delivery agents are inserted. The availability field indicates whether each agent is currently available.

4.9 Inserting Data into the Deliveries Table

The Deliveries table tracks the agent assigned to each order and the delivery status.

```
INSERT INTO Deliveries
```

```
(delivery_id, order_id, agent_id,
 pickup_time, delivery_time, delivery_status)
VALUES
(1, 1, 1, '2026-10-20 12:45:00',
 '2026-10-20 13:15:00', 'Delivered'),

(2, 2, 2, '2026-10-20 13:30:00',
 '2026-10-20 14:00:00', 'Delivered'),

(3, 3, 3, '2026-10-21 19:15:00',
 '2026-10-21 19:50:00', 'Delivered'),

(4, 4, 4, '2026-10-21 20:30:00',
 NULL, 'Picked Up');
```

Explanation: Four delivery records are inserted.

Orders 1, 2, and 3 have been delivered. Order 4 has been picked up but has not yet been delivered, so its delivery time is NULL.

Order 5 has no delivery record yet because it is still being prepared.

The UNIQUE constraint on `order_id` ensures that each order has at most one delivery record.

4.10 Inserting Data into the Payments Table

The Payments table stores the amount, payment method, payment status, and payment date.

```
INSERT INTO Payments
(payment_id, order_id, amount, payment_method,
 payment_status, payment_date)
VALUES
(1, 1, 320.00, 'UPI',
```

'Successful', '2026-10-20 12:31:00'),

(2, 2, 180.00, 'Card',

'Successful', '2026-10-20 13:16:00'),

(3, 3, 370.00, 'UPI',

'Successful', '2026-10-21 19:01:00'),

(4, 4, 360.00, 'Cash on Delivery',

'Pending', '2026-10-21 20:16:00'),

(5, 5, 220.00, 'Net Banking',

'Successful', '2026-10-22 10:31:00');

Explanation: Five payment records are inserted using valid payment methods and statuses.

The payment amounts are non-negative, and every payment references an existing order.

4.11 Inserting Data into the Reviews Table

The Reviews table stores customer ratings for restaurants and delivery services.

```
INSERT INTO Reviews
(review_id, order_id, customer_id,
 restaurant_rating, delivery_rating,
 comment, review_date)
VALUES
(1, 1, 1, 5, 5,
 'Excellent food and quick delivery',
 '2026-10-20 13:20:00'),
```

(2, 2, 2, 4, 4,

'Good food and service',

'2026-10-20 14:05:00'),

(3, 3, 3, 5, 4,

'Delicious pizza and good packaging',

'2026-10-21 19:55:00');

Explanation: Three customer reviews are inserted for completed orders.

Each review references an existing order and customer. Both ratings are within the permitted range of 1 to 5.

---

5. Summary of Sample Data

The following table summarizes the number of records inserted into each table.

Table Name	Number of Records
Customers	5
Addresses	6
Restaurants	5
MenuItems	10

Orders	5
OrderItems	8
DeliveryAgents	4
Deliveries	4
Payments	5
Reviews	3
Total	**55**

A total of **55 sample records** are inserted across the ten tables.

These records simulate customer registration, multiple delivery addresses, restaurant menus, food orders, payment processing, delivery assignments, and customer feedback.

6. Verification of Inserted Data

After inserting the records, the following SQL statements can be used to verify that the data has been stored correctly.

6.1 Display All Customers

SELECT * FROM Customers;

This query displays all registered customers.

6.2 Display All Restaurants

SELECT * FROM Restaurants;

This query displays the restaurants and their ratings.

6.3 Display All Menu Items

```
SELECT * FROM MenuItems;
```

This query displays the food items, prices, categories, and availability.

6.4 Display All Orders

```
SELECT * FROM Orders;
```

This query displays the order details and current order statuses.

6.5 Display All Payments

```
SELECT * FROM Payments;
```

This query displays the payment methods, amounts, and statuses.

6.6 Count Records in Each Table

```
SELECT 'Customers' AS Table_Name,
    COUNT(*) AS Total_Records
FROM Customers

UNION ALL

SELECT 'Addresses', COUNT(*)
FROM Addresses

UNION ALL

SELECT 'Restaurants', COUNT(*)
```

```
FROM Restaurants

UNION ALL

SELECT 'MenuItems', COUNT(*)
FROM MenuItems

UNION ALL

SELECT 'Orders', COUNT(*)
FROM Orders

UNION ALL

SELECT 'OrderItems', COUNT(*)
FROM OrderItems

UNION ALL

SELECT 'DeliveryAgents', COUNT(*)
FROM DeliveryAgents

UNION ALL

SELECT 'Deliveries', COUNT(*)
FROM Deliveries
```

```
UNION ALL

SELECT 'Payments', COUNT(*)
FROM Payments

UNION ALL

SELECT 'Reviews', COUNT(*)
FROM Reviews;
```

Expected Result: The query displays the number of records stored in each table. When all sample statements execute successfully in an empty database, the counts should match the summary in Section 5.

7. Verification of Relationships Using JOIN

The following query displays customer names, restaurants, orders, and payment details together.

```
SELECT
    c.full_name,
    r.restaurant_name,
    o.order_id,
    o.order_status,
    o.total_amount,
    p.payment_method,
    p.payment_status
FROM Orders o

JOIN Customers c
```

```
    ON o.customer_id = c.customer_id

JOIN Restaurants r
    ON o.restaurant_id = r.restaurant_id

LEFT JOIN Payments p
    ON o.order_id = p.order_id;
```

Explanation:

- The Customers table provides customer names.
- The Restaurants table provides restaurant names.
- The Orders table provides order status and total amount.
- The Payments table provides payment information.
- LEFT JOIN allows an order to appear even if a payment record does not exist.

Expected Result: Five order records should be displayed, with the associated customer, restaurant, order, and payment information.

8. Ensuring Data Integrity

The inserted sample records follow the database constraints defined in Assignment 5.

Primary Key Integrity: Each record uses a unique primary key.

Foreign Key Integrity: Every referenced customer, restaurant, address, menu item, order, and delivery agent exists in the corresponding parent table.

UNIQUE Constraint: Customer email addresses and phone numbers are unique. Each order has at most one delivery record, payment record, and review record.

CHECK Constraint: Menu prices and payment amounts are non-negative, quantities are positive, ratings are within their permitted ranges, and status and payment-method values are valid.

NOT NULL Constraint: Required fields contain values.

DEFAULT Constraint: Default values are available for fields such as order status, delivery status, payment status, and availability when those fields are omitted during insertion.

9. Result

Sample data was prepared for all ten tables of the Online Food Delivery System using SQL INSERT statements.

A total of 55 records were included to simulate customers, delivery addresses, restaurants, food items, orders, order items, delivery agents, deliveries, payments, and reviews.

The inserted records follow the defined primary key, foreign key, UNIQUE, CHECK, NOT NULL, and DEFAULT constraints.

Verification queries were provided to display the records, count the data in each table, and retrieve related information using JOIN operations.

10. Conclusion

This assignment demonstrates how sample data can be inserted into a relational database using SQL. The Online Food Delivery System was populated with realistic records representing the major activities of a food delivery application.

The sample data illustrates relationships between customers, addresses, restaurants, menu items, orders, payments, deliveries, and reviews. Proper ordering of INSERT statements ensures that foreign key dependencies are satisfied.

The verification queries help confirm that the records are stored correctly and can be retrieved through SQL operations.

Therefore, sample data insertion is an important step in testing database design, verifying relationships, and preparing a database for further query implementation and application development.